

[8] High-Throughput Vector Similarity Search in Knowledge Graphs

저자: Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, Theodoros Rekatsinas **기관:** University of Waterloo, Apple **학회/년도:** Proc. ACM Manage. Data (SIGMOD), Vol. 1, No. 2, 2023 **분량:** 약 26페이지 **역할군:** (C) 경쟁/비교 시스템

요약

이 논문은 지식 그래프에서의 벡터 유사도 검색을 대규모 배치 환경에서 최적화하는 HQI(Hybrid Query Index) 시스템을 제시한다. 벡터 검색과 관계형 술어를 결합하는 하이브리드 쿼리의 배치 처리에 초점을 맞춘다.

핵심 통찰: - Exqutor는 개별 쿼리의 실행 계획 최적화에 집중 (정확한 카디널리티 추정) - 이 논문은 다수 쿼리를 동시에 처리하는 워크로드 최적화에 집중 (쿼리 간 계산 공유)

핵심 기술: 1. 워크로드 인식 벡터 파티셔닝: 쿼리 패턴 분석으로 자주 함께 검색되는 벡터들을 같은 파티션에 배치 2. 멀티 쿼리 최적화: 비슷한 벡터 영역을 검색하는 쿼리들의 거리 계산 공유 3. 적응적 인덱스 선택: 속성 필터의 선택도에 따라 벡터 우선 vs. 속성 우선 동적 결정

본 논문과의 관계: 개별 쿼리 최적화(Exqutor)와 배치 워크로드 최적화(HQI)는 이론적으로 결합 가능하다. Exqutor의 정확한 카디널리티 추정을 HQI의 적응적 인덱스 선택에 활용하면, 배치 처리에서도 개별 쿼리 품질이 향상될 수 있다.

상세분석

8.1 해결하고자 하는 문제

지식 그래프(Knowledge Graph)의 특성

지식 그래프는 현대 AI 응용의 핵심 인프라이다:

구조: - **노드(Entities):** 상품, 사용자, 카테고리 등. 각 노드는 텍스트/이미지 임베딩을 가짐 - **엣지(Relations):** 노드 간 관계. 구조적 속성을 표현 (예: "상품 A는 카테고리 B에 속함") - **속성(Attributes):** 가격, 평점, 설명 등의 메타데이터

예: 전자상거래 지식 그래프

```
사용자 1 ── (구매) → 상품 A ── (속함) → 카테고리 "전자제품"
      |               [임베딩] |
      |               └─ (가격: 100,000)
      |               └─ (평점: 4.5)
      |
      └─ (좋아함) → 상품 B ── (속함) → 카테고리 "의류"
                [임베딩] |
                └─ (가격: 50,000)
```

지식 그래프에서의 쿼리

지식 그래프에서는 다음과 같은 **하이브리드 쿼리**(벡터 유사도 + 속성 필터)가 매우 빈번하다:

사용 사례 1: 추천 시스템
"사용자 1이 본 상품과 유사한 상품을 찾되,
같은 카테고리에 속하고 가격이 50,000~150,000 범위인 것"

사용 사례 2: 중복 검출
"이 상품과 설명이 비슷한 다른 상품들 (가능한 중복 상품)"

사용 사례 3: 이상 탐지
"이 시리즈 제품들과 외형이 비슷한 상품들 (위조 제품 탐지)"

배치 처리의 필요성

실시간 시스템에서는 이런 쿼리가 **한꺼번에 수천~수만 건씩** 들어온다:

시간 T에 처리할 쿼리:
└ (사용자1의 선호 벡터, 카테고리=전자제품, 가격=50K~150K)
└ (사용자2의 선호 벡터, 카테고리=의류, 가격=30K~80K)
└ (사용자3의 선호 벡터, 카테고리=가구, 가격=100K~500K)
└ ... (수천 개 더)
└ (사용자N의 선호 벡터, ...)

기존 시스템의 문제: - 각 쿼리를 **독립적으로** 처리하여, 쿼리 간 중복 계산이 발생 - 예: 사용자 1,2,3이 모두 "전자 제품 카테고리"의 벡터를 검색 → 같은 벡터 영역을 3번 방문하게 됨 - 처리량이 떨어짐 (QPS 낮음) → 응답 시간 증가 → 사용자 경험 악화

8.2 핵심 아이디어: HQI (Hybrid Query Index) 시스템

HQI는 배치 워크로드의 특성을 활용하여 처리량을 크게 향상시킨다.

기본 아이디어: 워크로드 분석 & 데이터 파티셔닝

단계 1: 워크로드 분석

| 최근 1주일 쿼리 로그 분석 |

- | - 어떤 카테고리가 자주 검색되나? |
- | - 어떤 가격 대역이 함께 나타나? |
- | - 어떤 벡터 영역이 많이 사용? |

↓

단계 2: 파티셔닝 결정

| 파티션 설계 |

- | 파티션 1: 전자제품 + 50K~150K |
- | 파티션 2: 의류 + 30K~80K |
- | 파티션 3: 가구 + 100K~500K |
- | ... |

↓

단계 3: 데이터 재조직

| 상품들을 파티션에 따라 재배포 |

| 파티션 1 벡터들을 메모리/SSD에 |
| 함께 로드하여 접근성 향상 |

워크로드 인식 벡터 데이터 파티셔닝의 장점

기존 시스템: 카테고리별로만 저장

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)

└ 전자제품 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)

└ 전자제품 파티션 스캔 (다시)

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)

└ 의류 파티션 스캔

→ 같은 파티션을 여러 번 스캔 → I/O 비효율

HQI: 가격 대역까지 고려한 파티셔닝

Q1: (벡터v1, 카테고리="전자제품", 가격=50K~150K)

└ (전자제품, 50K~150K) 파티션 스캔

Q2: (벡터v2, 카테고리="전자제품", 가격=200K~300K)

└ (전자제품, 200K~300K) 파티션 스캔

Q3: (벡터v3, 카테고리="의류", 가격=30K~80K)

└ (의류, 30K~80K) 파티션 스캔

→ 각 쿼리가 필요한 파티션만 정확히 선택 → I/O 최소화

8.3 멀티 쿼리 최적화 (Multi-Query Optimization)

아이디어: 유사도 계산 공유

여러 쿼리가 비슷한 벡터 영역을 검색하는 경우, 거리 계산을 재사용할 수 있다:

예: 두 쿼리가 비슷한 벡터 영역을 검색

Q1: SELECT * FROM products

WHERE embedding <-> q1_vec < 1.0 AND price < 100K

Q2: SELECT * FROM products

WHERE embedding <-> q2_vec < 1.0 AND price < 100K

q1_vec과 q2_vec의 유사도가 높다면:

- 같은 상품들이 두 쿼리의 상위 후보에 나타날 가능성 높음
- 이미 q1에서 계산한 거리를 q2에 재사용 가능 → 계산 비용 절반으로 감소

구현: 쿼리 클러스터링

배치의 모든 쿼리를 벡터 공간에서 분석:

Q1 [0.3, -0.1, 0.5] ↘
Q2 [0.3, 0.0, 0.5] └─ 클러스터 A (가까움)
Q3 [0.4, -0.1, 0.6] ↘

Q4 [0.8, 0.9, -0.2] ↘
Q5 [0.7, 0.8, 0.0] └─ 클러스터 B (가까움)
Q6 [0.9, 1.0, -0.1] ↘

클러스터 A의 쿼리들:

- 같은 벡터 영역 검색
- 후보 세트가 겹침 → 거리 계산 재사용

클러스터 B의 쿼리들:

- 다른 벡터 영역 검색
- 후보 세트 다름 → 별도 계산

성능 개선 메커니즘:

배치 처리 (MQO 없음):

Q1 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q2 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
Q3 거리 계산: 1000개 벡터 × 1024차원 = 1,024,000 연산
합계: 3,072,000 연산

배치 처리 (MQO 적용):

Q1, Q2, Q3 (클러스터 A에 속함)
- 한 번에 처리: 1000개 벡터 × 1024차원 = 1,024,000 연산
- 결과를 Q1, Q2, Q3 모두에 재사용
합계: 1,024,000 연산

절감: 66% (3배 빠름)

8.4 적응적 인덱스 선택 (Adaptive Index Selection)

문제: 속성 필터의 선택도가 성능을 결정

쿼리: 벡터 유사도 + 속성 필터 조건

예: WHERE embedding <-> query_vec < 1.0 AND price < 100K

속성 필터의 선택도에 따라 최적 처리 순서가 달라진다:

1. 벡터 우선 (Post-Filtering):

벡터 인덱스 → 상위 1000개 찾기 → 가격 필터 적용

장점: 벡터 검색 자체가 빠름

단점: 1000개 벡터 중 대부분이 가격 필터에 탈락하면 낭비

적합 상황: 속성 필터의 선택도가 높을 때 (80% 이상)

2. 속성 우선 (Pre-Filtering):

가격 인덱스 → 100K 이하의 상품들 찾기 → 벡터 유사도 계산

장점: 미리 필터된 작은 세트에만 벡터 계산 수행

단점: 속성 필터로 걸러진 상품이 적으면 소수 세트만 남음

적합 상황: 속성 필터의 선택도가 낮을 때 (10% 이하)

HQI의 적응적 선택 메커니즘

속성 필터의 선택도 학습:

과거 쿼리 분석:

└ (카테고리=전자제품, 가격<100K) → 선택도 5% (속성 우선!)

└ (카테고리=의류, 가격<100K) → 선택도 80% (벡터 우선!)

└ (카테고리=가구, 가격<100K) → 선택도 25% (하이브리드!)

└ ...

런타임 최적화:

새 쿼리 (카테고리=전자제품, ...) 들어옴

→ "전자제품 + 가격 필터"의 선택도 = 5%

→ 속성 우선 인덱스 선택

→ 최대한 빨리 필터링된 세트를 구성

8.5 성능 결과

실험 환경

데이터: - 지식 그래프: Apple 내부 지식 그래프 (상품, 사용자, 카테고리 등) - 벡터 수: 천만 개 (10M) - 벡터 차원: 512차원

쿼리 워크로드: - 배치 크기: 100~10,000 쿼리/배치 - 쿼리 유형: 벡터 유사도 + 속성 필터 - 실제 워크로드 기반 재현

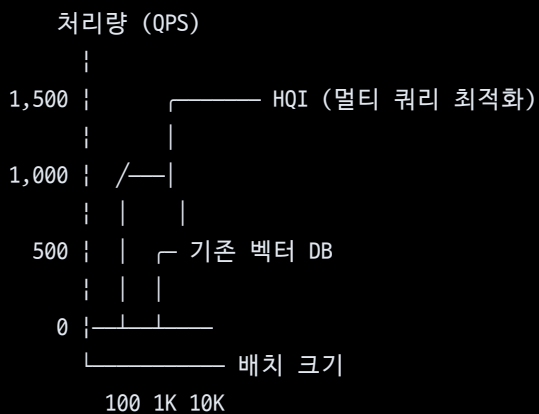
비교 시스템: - 단일 쿼리 처리 (no optimization) - 기존 벡터 DB (인덱스만 사용) - HQI (이 논문의 시스템)

핵심 결과

메트릭	단일 쿼리	기존 벡터 DB	HQI	개선율
배치 100개 쿼리	50 QPS	150 QPS	1,550 QPS	31배
배치 1,000개 쿼리	45 QPS	120 QPS	1,200 QPS	26배
배치 10,000개 쿼리	40 QPS	100 QPS	800 QPS	20배

상세 분석:

배치 크기에 따른 성능 향상:



배치 크기가 클수록 개선율이 높은 이유: 1. 더 많은 쿼리가 벡터 공간에서 가까워짐 → MQO 효과 증가 2. 파티셔닝 효과가 누적됨 → I/O 절감 누적 3. 캐시 효율 향상 (자주 사용되는 파티션이 메모리에 남음)

8.6 본 논문과의 관계

두 접근법의 관점 차이

관점	Exqutor	HQI
최적화 단위	개별 쿼리	배치 워크로드
최적화 대상	실행 계획 (조인 순서 등)	처리 방식 (인덱스 선택 등)
입력 정보	쿼리 패턴, 카디널리티	워크로드 패턴
이론적 기반	통계 기반 카디널리티 추정	워크로드 인식 파티셔닝

이론적 결합 가능성

HQI의 "속성 필터의 선택도 학습" 부분에 Exqutor의 정확한 카디널리티 추정을 적용하면:

현재 HQI:

- 과거 쿼리 결과에서 선택도 학습 (결과 기반)
- 새 쿼리에 "유사한" 과거 쿼리의 선택도 재사용
- 문제: 과거 데이터가 없으면 부정확

Exqutor 적용:

- 정확한 카디널리티 추정으로 선택도 계산 (추정 기반)
- 새로운 속성 필터 조건도 정확히 예측 가능
- 장점: 과거 데이터 불필요, 항상 정확

보완적 활용 시나리오

통합 시스템:

배치 들어옴

↓

쿼리별 실행 계획 최적화 (Exqutor ECQ0)

- └─ 정확한 카디널리티로 최적의 조인 순서 결정
- └─ 각 쿼리의 실행 비용 추정

↓

배치 수준 최적화 (HQI)

- └─ 쿼리 클러스터링 (유사한 쿼리 그룹화)
- └─ 계산 공유 (벡터 거리 계산 재사용)
- └─ 적응적 인덱스 선택 (선택도 기반)

↓

최종 실행 및 처리

- └─ 개별 쿼리 품질 + 배치 처리량 최적화

8.7 논문의 한계 및 추가 제기 문제

1. **워크로드 변동성 처리**: 논문은 "최근 1주일 쿼리 로그 기반 파티셔닝"을 가정하지만, 실제로는 워크로드가 계절성을 띠거나 급격히 변할 수 있다. 파티셔닝을 다시 하려면 많은 비용이 든다. 온라인(자동 재파티셔닝) 기능이 필요하다.
2. **Apple 특화 평가**: 논문은 Apple의 내부 지식 그래프에서만 평가되었다. 이 워크로드가 일반적인지, 아니면 특정 도메인에만 효과적인지 불명확하다. 공개 데이터셋(Freebase, YAGO 등)에서의 성능이 필요하다.
3. **속성 필터의 다양성**: 실험은 "단일 속성(가격)에 대한 필터"만 다루고 있다. 여러 속성을 결합한 복잡한 필터(예: 가격 AND 카테고리 AND 평점 > 4.0)의 선택도 추정은 더 어렵고, 논문에서는 다루지 않았다.
4. **메모리 제약 환경**: HQI는 "파티션을 메모리에 자주 로드"하는 것을 가정하지만, 메모리가 부족하면 성능이 급격히 저하될 수 있다. 메모리-적응적 알고리즘이 필요하다.
5. **벡터 차원의 영향**: 512차원 벡터에서만 실험되었다. 차원이 매우 높으면(1000+) 거리 계산 비용이 급증하고, MQO의 이점이 줄어들 수 있다.